AI WeatherWise

Intelligent Weather Forecasts and AI-Powered Weather Insights

---

Phase 3 – Project Design

Technical Design and System Architecture Documentation

Project type: AI-powered weather intelligence REST API / backend

Stack: Node.js · Express.js · MongoDB · Mongoose · JWT · bcrypt.js · Google Gemini AI · OpenWeatherMap API · Postman / Thunder Client

BSc Computer Science Project

TABLE OF CONTENTS

1. Technical Architecture

AI WeatherWise follows a layered (n-tier) REST architecture built on the Model-View-Controller pattern. Each request passes through a fixed chain of layers, and every layer has one job, which keeps the code easy to test, debug and extend. The backend is a headless service, so the "view" layer is the API routing layer that links each request path to its controller. Weather requests take a branch from the controller layer to an external weather service (OpenWeatherMap), and AI requests take a branch from the controller layer to Google Gemini. The backend runs on port 5001.

1.1 Request Flow

Standard flow: Client / API Testing Tool → Express.js Server → API Routes → Authentication Middleware (protected routes) → Controllers → Services / Business Logic → Mongoose Models → MongoDB Atlas → JSON Response.

Weather flow: Client → Weather route (GET /api/weather/:city) → Weather Controller → Weather Service → OpenWeatherMap API → Weather Data → JSON Response. This route is public and needs no JWT. If the external weather API key is not configured, the service returns a structured fallback (mock) response flagged with isMock.

AI flow: Client → (authenticated request where required) → AI route (POST /api/ai/weather-recommendation) → AI Controller → Weather details (city, temperature, humidity, condition) → Gemini AI Service → Google Gemini API → AI Weather Summary / Recommendation → JSON Response.

Favorite Location flow: User → Location operation → Authentication Middleware (JWT) → Location Controller → Location Model (Mongoose) → MongoDB Atlas → Location Response.

1.2 Purpose of Each Component

Component	Purpose
Client / API Testing Tool	Postman or Thunder Client sends HTTP requests (with JSON bodies and, for protected routes, the JWT in the Authorization header) and displays the JSON responses.
Express.js Server	Entry point of the backend. Loads environment variables (PORT, MONGO_URI, JWT_SECRET, GEMINI_API_KEY), connects to MongoDB, applies global middleware (JSON body parsing, CORS), mounts the routes and listens on port 5001.
API Routes	Map each URL and HTTP method (e.g. POST /api/auth/login, GET /api/weather/:city) to the correct controller function. Routes are grouped by feature: authentication, location, weather and AI.

Component	Purpose
Authentication Middleware	Reads the Bearer token from the request header and verifies it with the JWT secret. It separates public routes (such as weather lookups) from private, JWT-protected routes; requests without a valid token are rejected with 401.
Controllers	The Auth, Location, Weather and AI controllers handle the request and response cycle: read input, apply request constraints, delegate to services and models, and return standard JSON responses.
Services / Business Logic	Hold the logic that talks to external systems, so controllers stay thin. The Weather Service communicates with OpenWeatherMap and the AI Service communicates with Google Gemini. Password hashing with bcrypt.js is part of the authentication logic.
Weather Service	Requests current weather metrics (temperature, humidity, wind speed and condition) for a city from OpenWeatherMap and returns them to the controller; provides the fallback (mock) response when no API key is configured.
Gemini AI Service	Builds a prompt from the supplied weather details, calls the Gemini API through the @google/genai SDK, and returns the model output as a weather summary or personalized recommendation.
Mongoose Models	Define the User and Location schemas, validation rules, timestamps, the Location → User reference and the compound unique index on { user, city, country }.
MongoDB Atlas	Cloud-hosted database (connected through MONGO_URI) that persistently stores the Users and Locations collections.
External Weather API (OpenWeatherMap)	External source of live current-weather data for the requested city.
Google Gemini API	External generative AI that produces natural-language weather summaries and personalized, condition-aware recommendations.

1.3 Technical Architecture Diagram

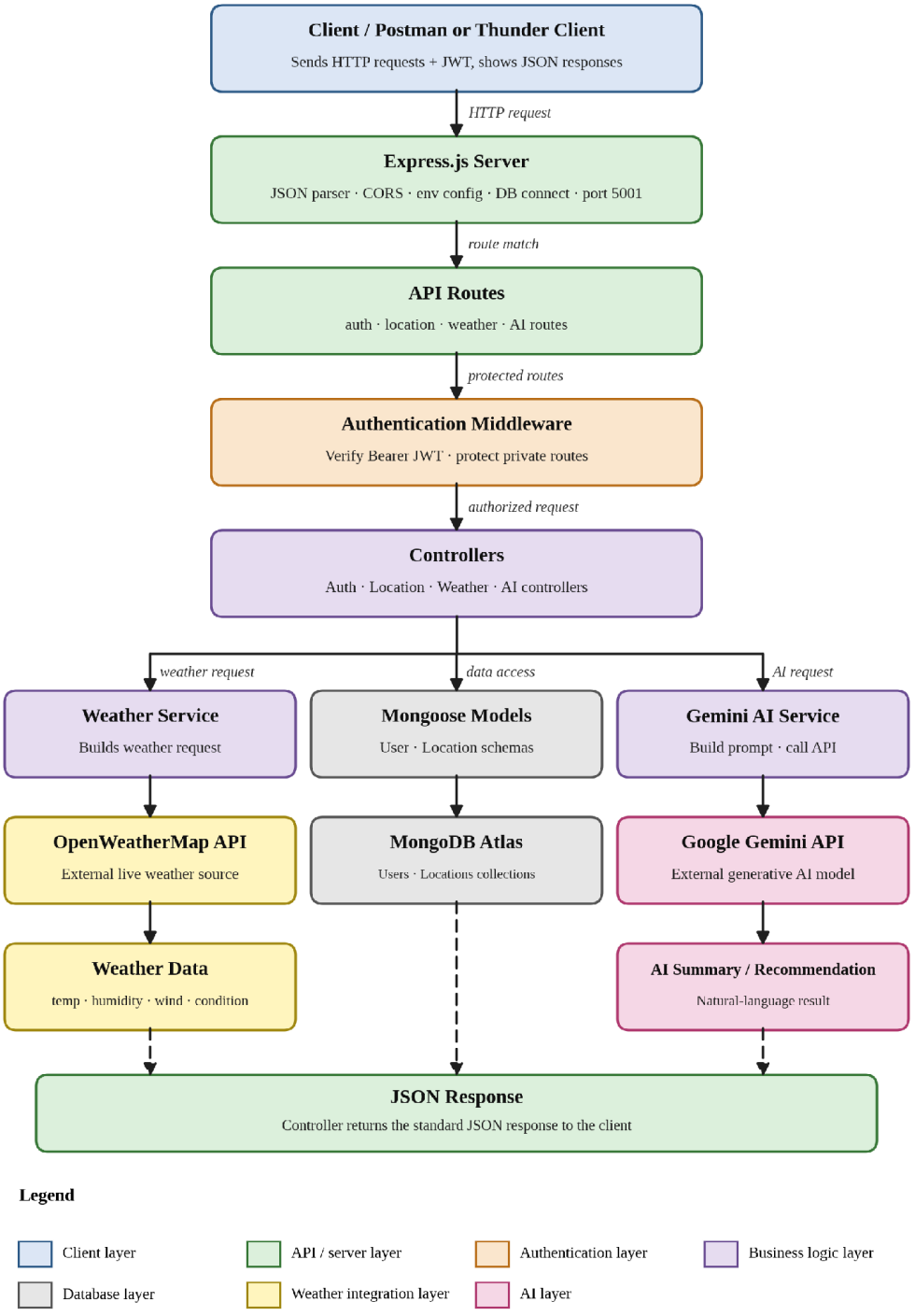


Figure 1: AI WeatherWise – Technical Architecture Diagram (solid arrows: request flow; dashed arrows: results returning to the controller and the JSON response)

2. ER Diagram

The database has two collections: User and Location. No other entities are implemented in the current project.

2.1 Relationship

One User can have many Locations (USER 1 ——— N LOCATION). Each Location, stored as a favorite location, belongs to exactly one User. The Location.user field stores the _id of the owning User (a Mongoose ObjectId with ref: "User"), and it is populated with the authenticated user's id when a favorite is created. This reference is what scopes every user to viewing and changing only their own favorite locations. A compound unique index on { user, city, country } also prevents a user from saving the same city and country twice.

2.2 Entity Definitions

USER

Entity	Attribute	Key / Type	Notes
USER	_id	PK · ObjectId	Auto-generated by MongoDB
	name	String	Required
	email	String	Required, unique; stored in lowercase and validated against an email pattern
	password	String	Required; stored as a bcrypt hash; minimum 6 characters before hashing
	createdAt	Date	Timestamp (Mongoose timestamps)
	updatedAt	Date	Timestamp (Mongoose timestamps)

LOCATION (Favorite Location)

Entity	Attribute	Key / Type	Notes
LOCATION	_id	PK · ObjectId	Auto-generated by MongoDB
	city	String	Required; name of the saved favorite city
	country	String	Required; country of the saved city
	user	FK · ObjectId	Required; ref → User._id (owning user)
	createdAt	Date	Timestamp (Mongoose timestamps)
	updatedAt	Date	Timestamp (Mongoose timestamps)

PK = Primary Key; FK = Foreign Key / Reference. The relationship is Location.user → User._id. The implemented User schema contains no role field; the roles described in Section 4 are functional responsibilities.

2.3 Entity Relationship Diagram

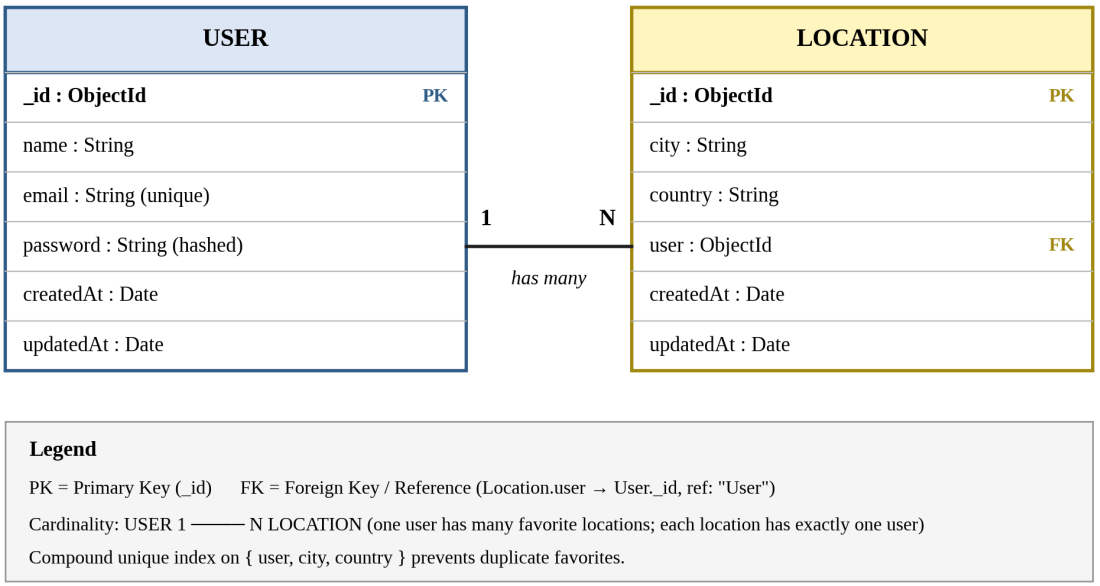


Figure 2: AI WeatherWise – Entity Relationship Diagram

3. Features

The following features make up the scope of AI WeatherWise as documented in the project documentation. Endpoint paths are shown only where the project's API testing evidence confirms them; the favorite-location paths are not specified in the source and are therefore not listed.

3.1 Feature Summary Table

Feature	Purpose	Main Component
User Registration	Creates an account from name, email and password after input validation (POST /api/auth/register).	Auth Controller
User Login	Verifies email and password and returns a signed JWT (POST /api/auth/login).	Auth Controller
JWT Authentication	Provides stateless, token-based access to protected routes.	Authentication Middleware
Secure Password Hashing	Hashes passwords with bcrypt.js before storage and verifies them on login.	bcrypt.js in Auth Controller
Protected User Access	Separates public routes from private routes and keeps user data scoped to the authenticated user.	Authentication Middleware; API Routes
Weather Information Retrieval	Fetches current weather metrics (temperature, humidity, wind speed, condition) for a city (GET /api/weather/:city).	Weather Controller; Weather Service
Public Weather Access	Allows casual weather lookups without creating an account.	Weather Routes
Resilient Fallback (Mock) Mode	Keeps the weather endpoint returning a structured response, flagged with isMock, when no external API key is configured.	Weather Service
Create Favorite Location	Saves a city and country to the authenticated user's favorites.	Location Controller; Location Model
View Favorite Locations	Retrieves the authenticated user's saved favorite locations.	Location Controller
Update Favorite Location	Updates an existing favorite location after an ownership check.	Location Controller
Delete Favorite Location	Removes a favorite location after an ownership check.	Location Controller
Duplicate Favorite Prevention	Blocks the same city and country from being saved twice by one user.	Location Model (compound unique index)

Feature	Purpose	Main Component
AI Weather Recommendation	Generates a personalized, condition-aware recommendation such as clothing or hydration advice (POST /api/ai/weather-recommendation).	AI Controller; Gemini AI Service
AI Weather Summary	Turns raw weather metrics into a natural-language summary through the same AI Service integration.	Gemini AI Service
Validation	Enforces required fields, email format, minimum password length and required city and country.	Mongoose Models; Controllers
Error Handling	Standardizes how errors are surfaced across the API as JSON.	Centralized Error Handler
Request Sanitization	Sanitizes incoming request data as part of request processing.	Request processing layer
CORS Support	Handles cross-origin requests at the server gateway.	Express.js Server

Note: a separately tested endpoint for AI weather summaries is not shown in the source material; summary generation is described as part of the same AI Service integration that produces recommendations.

4. Roles and Responsibilities

4.1 System User Role: Registered User

Anyone can register and log in. Once authenticated with a valid JWT, the registered user can perform the actions below, and favorite locations are always scoped to that user. Weather lookups are also available as public API access without an account; this is an access route, not a separate role.

- Register an account and log in
- Send the JWT with requests to protected routes
- Create, view, update and delete their own favorite locations
- Request real-time weather information for a city
- Request AI weather summaries and personalized recommendations
- Manage their own authentication credentials, such as updating profile details

Administrator: The project documentation also describes an Administrator responsible for user account management (creating, deleting or suspending accounts), oversight of API health and system logs, and monitoring database integrity. The implemented User schema has no role field and no administrator endpoints are documented, so these are documented responsibilities rather than confirmed endpoints.

4.2 Component and Role Responsibilities

Component / Role	Responsibility
Registered User	Registers, logs in, keeps the JWT, manages favorite locations, requests weather information and requests AI weather insights.
Administrator (documented)	Account management, oversight of API health and system logs, and monitoring of database integrity, as described in the project documentation.
Express.js Server	Starts the backend on port 5001, parses JSON, handles CORS, connects to MongoDB and mounts the routes.
API Routes	Map endpoints and HTTP methods to controllers and attach authentication to private routes.
Authentication Middleware	Verifies the Bearer JWT and separates public routes from JWT-protected private routes.
Controllers	Handle requests and responses, apply validation and constraints, delegate to services and models, and return JSON.
Services	Weather Service calls OpenWeatherMap; Gemini AI Service calls Google Gemini through @google/genai.

Component / Role	Responsibility
Mongoose Models	Define and validate the User and Location schemas and the Location → User reference.
MongoDB Atlas	Persists users and favorite locations.
Weather API (OpenWeatherMap)	Supplies live current-weather data for the requested city.
Gemini AI Service	Builds prompts from weather details and returns AI-generated summaries and recommendations.
Error Handler	Centrally standardizes how errors are returned to the client as JSON.

5. User Flow

The flow below traces a typical journey of a WeatherWise user, from first access to receiving a response.

5.1 Step-by-Step Flow

1. The user opens Postman or Thunder Client and sends requests to the AI WeatherWise backend on port 5001. Weather lookups need no account; personalized features do.
2. A new user calls the registration endpoint (POST /api/auth/register) with name, email and password.
3. The server validates the input: required fields, email format and a minimum password length of 6 characters.
4. The password is hashed with bcrypt.js before it is stored.
5. The user is saved in MongoDB Atlas through the Mongoose User model, and the server responds with 201 Created.
6. The user logs in (POST /api/auth/login) with email and password.
7. The server looks up the user and verifies the submitted password against the stored hash with bcrypt.compare.
8. On success, a signed JWT is generated and returned with the user object (200 OK).
9. The user sends the token in the Authorization: Bearer <token> header with every protected request.
10. The authentication middleware verifies the token. Invalid or missing tokens are rejected with 401 Unauthorized; valid requests continue to the controller.
11. The user chooses a WeatherWise operation: weather information, favorite location management, or an AI weather insight.
12. For weather information, the Weather Controller and Weather Service request current conditions from OpenWeatherMap (GET /api/weather/:city) and return city, temperature, humidity, windSpeed and condition, or fallback data flagged with isMock when no API key is configured.
13. For favorite locations, the Location Controller creates, views, updates or deletes the user’s locations through the Location model in MongoDB Atlas. Updates and deletes pass an ownership check, and duplicate city and country pairs are blocked.
14. For an AI weather insight, the user sends the weather details (city, temperature, humidity, condition) to POST /api/ai/weather-recommendation, and the AI Controller passes them to the Gemini AI Service.

15. The Gemini AI Service sends the details to the Google Gemini API, which processes them and produces an AI weather summary or personalized recommendation.
16. The AI-generated result is returned to the controller, and the API sends it to the client as a standard JSON response (200 OK).
17. Any error is handled by the centralized error handler, and every response, successful or not, is returned as JSON.

5.2 User Flow Diagram

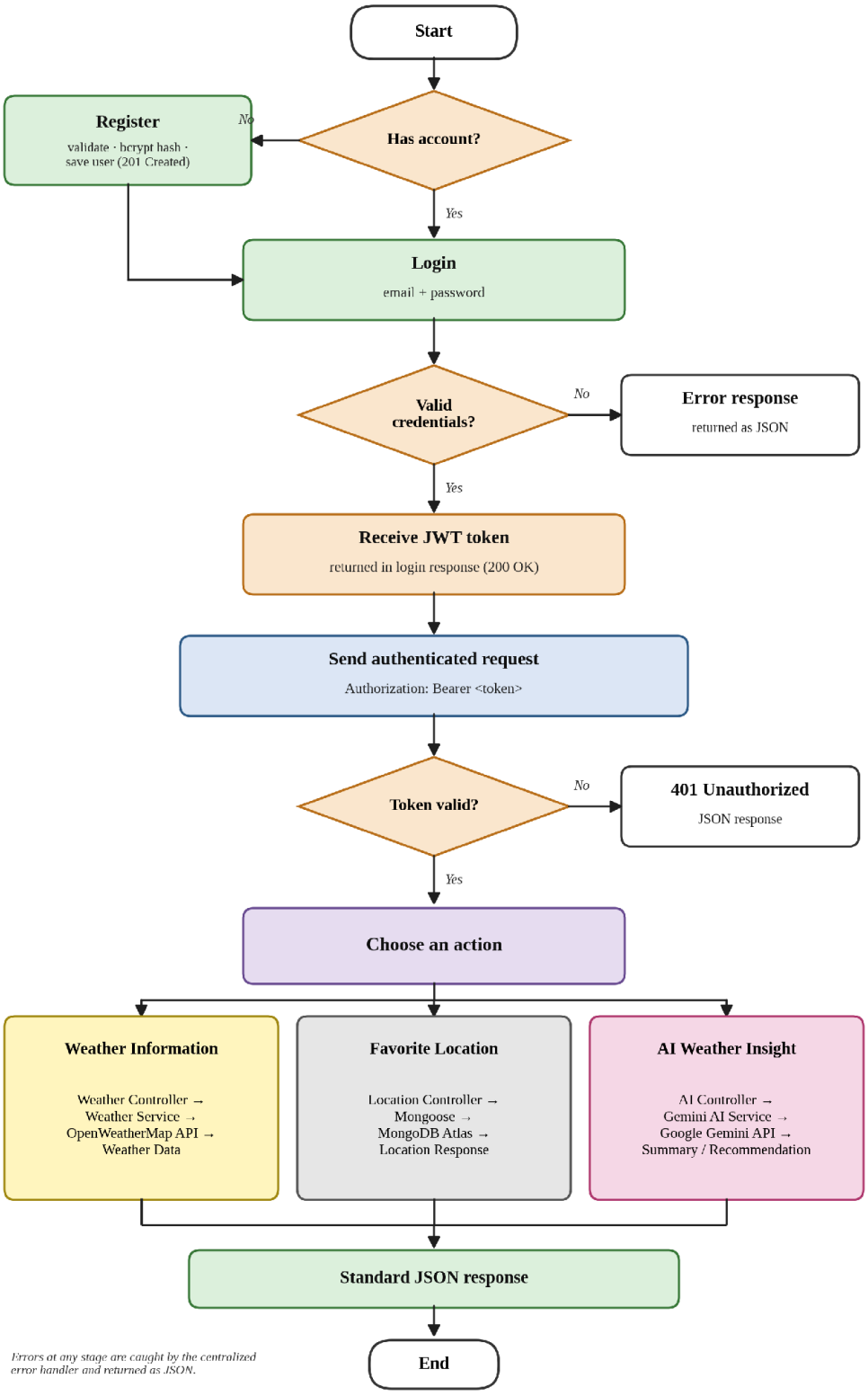


Figure 3: AI WeatherWise – User Flow Diagram